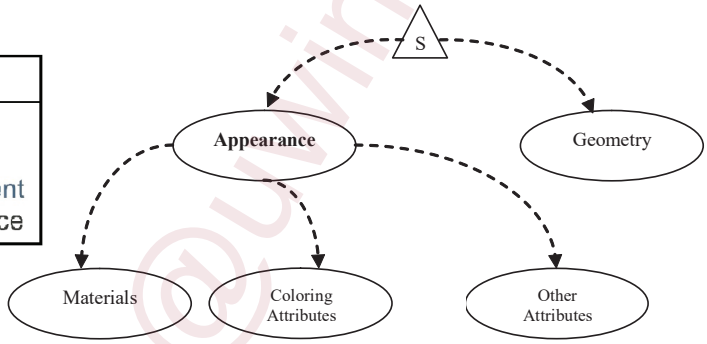
PART I. DEFINING OBJECT APPEARANCE

1. The color, texture, and material of shapes determine how they look like, which is specified through the associated appearance in Java 3D

   - `Appearance` class works together with `Geometry` as the subclasses of `NodeComponent` to determine the properties of `Shape3D`.

Class Hierarchy

```
java.lang.Object
  └ org.jogamp.java3d.SceneGraphObject
      └ org.jogamp.java3d.NodeComponent
          └ org.jogamp.java3d.Appearance
```



- `Appearance` attributes are subclasses of `NodeComponent`
  - Rendering control: `PointAttributes`, `LineAttributes`, `PolygonAttributes`, and `RenderingAttributes`
  - Color and transparency control: `ColoringAttributes`, `TransparencyAttributes`, and `Material`
  - Texture control: `Texture`, `TextureAttributes`, and `TexCoordGeneration`
- Default values of `Appearance` using constructor

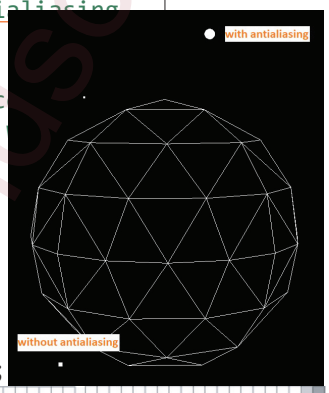
```
Appearance appear = new Appearance ()
```

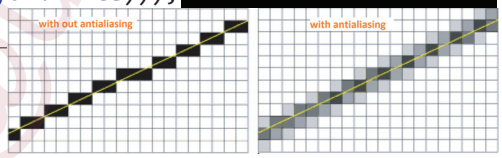
	Default values
Points and lines	Size and width of 1 pixel, no antialiasing
Intrinsic color	White
Transparency	Disabled
Depth buffer	Enabled, read and write accessible

2. Attributes of Appearance

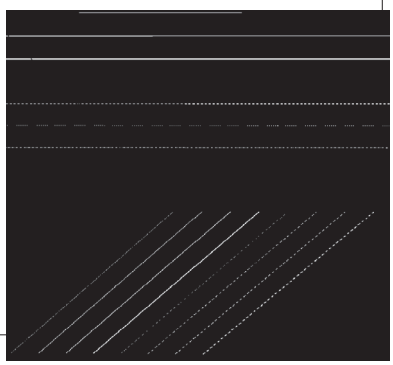
- Rendering control

```
private Shape3D appPoint(float x, float y, float z, float size, boolean tag) {
    PointArray pt = new PointArray(1, GeometryArray.COORDINATES);
    pt.setCoordinate(0, new Point3f(x, y, z));     // set location of the point

    Appearance app = new Appearance();             // 'tag' for antialiasing
    PointAttributes pointAtt = new PointAttributes(size, tag);

    app.setPointAttributes(pointAtt);               // set appearanc
    return new Shape3D(pt, app);                    // create point
}

/* a function to attach objects to scene_TG */
private void createContent(TransformGroup scene_TG) {
    scene_TG.addChild(appPoint(0.3f, 0.4f, 0.0f, 4f, false));
    scene_TG.addChild(appPoint(0.4f, -0.3f, 0.0f, 8f, false));
    scene_TG.addChild(appPoint(-0.2f, 0.6f, 0.0f, 20f, true));
    scene_TG.addChild(new Sphere(0.4f, fun.geoPlaneApp(fun.White)));
}
```

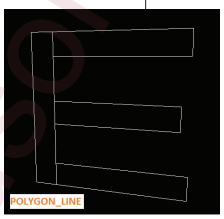




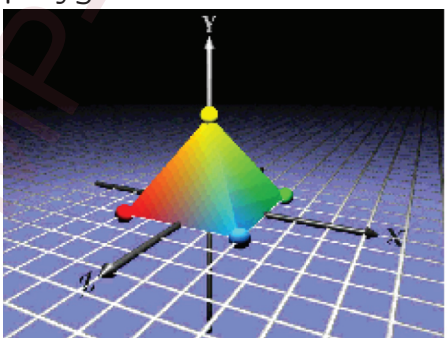

```
private void createContent(TransformGroup scene_TG) {
    scene_TG.addChild(aLine(-0.8f, 0.8f, 0.8f, 0.8f, 1f, LineAttributes.PATTERN_SOLID, false));
    scene_TG.addChild(aLine(-0.8f, 0.7f, 0.8f, 0.7f, 3f, LineAttributes.PATTERN_SOLID, false));
    scene_TG.addChild(aLine(-0.8f, 0.6f, 0.8f, 0.6f, 5f, LineAttributes.PATTERN_SOLID, false));
                                                    // test line pattern
    scene_TG.addChild(aLine(-0.8f, 0.4f, 0.8f, 0.4f, 3f, LineAttributes.PATTERN_DASH, false));
    scene_TG.addChild(aLine(-0.8f, 0.3f, 0.8f, 0.3f, 3f, LineAttributes.PATTERN_DOT, false));
    scene_TG.addChild(aLine(-0.8f, 0.2f, 0.8f, 0.2f, 3f, LineAttributes.PATTERN_DASH_DOT, false));
                                                    // test antialiasing
    scene_TG.addChild(aLine(-0.8f, -0.8f, -0.2f, -0.1f, 1f, LineAttributes.PATTERN_SOLID, false));
    scene_TG.addChild(aLine(-0.7f, -0.8f, -0.1f, -0.1f, 1f, LineAttributes.PATTERN_SOLID, true));
    scene_TG.addChild(aLine(-0.6f, -0.8f, 0.0f, -0.1f, 3f, LineAttributes.PATTERN_SOLID, false));
    scene_TG.addChild(aLine(-0.5f, -0.8f, 0.1f, -0.1f, 3f, LineAttributes.PATTERN_SOLID, true));
    scene_TG.addChild(aLine(-0.4f, -0.8f, 0.2f, -0.1f, 1f, LineAttributes.PATTERN_DASH, false));
    scene_TG.addChild(aLine(-0.3f, -0.8f, 0.3f, -0.1f, 1f, LineAttributes.PATTERN_DASH, true));
    scene_TG.addChild(aLine(-0.2f, -0.8f, 0.4f, -0.1f, 3f, LineAttributes.PATTERN_DASH, false));
    scene_TG.addChild(aLine(-0.1f, -0.8f, 0.5f, -0.1f, 3f, LineAttributes.PATTERN_DASH, true));
}
private Shape3D aLine(float x1, float y1, float x2, float y2,
        float width, int ptn, boolean tag) {
    LineArray line = new LineArray(2, GeometryArray.COORDINATES);
    line.setCoordinate(0, new Point3f(x1, y1, 0.0f));
    line.setCoordinate(1, new Point3f(x2, y2, 0.0f));

    Appearance app = new Appearance();
    LineAttributes lineAtt = new LineAttributes(width, ptn, tag);
    app.setLineAttributes(lineAtt);

    return new Shape3D(line, app); // create line with appearance
}
```

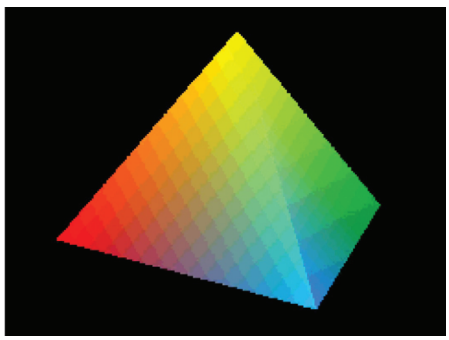


```
private Shape3D QuadE() {
    Point3f[] pts = { new Point3f(-0.5f, -0.5f, 0.0f), new Point3f(-0.35f, -0.5f, 0.0f),
            new Point3f(0.5f, -0.5f, 0.0f), new Point3f(-0.35f, -0.35f, 0.0f),
            new Point3f(0.5f, -0.35f, 0.0f), new Point3f(-0.35f, -0.085f, 0.0f),
            new Point3f(0.45f, -0.085f, 0.0f), new Point3f(-0.35f, 0.065f, 0.0f),
            new Point3f(0.45f, 0.065f, 0.0f), new Point3f(-0.35f, 0.35f, 0.0f),
            new Point3f(0.5f, 0.35f, 0.0f), new Point3f(-0.5f, 0.5f, 0.0f),
            new Point3f(-0.35f, 0.5f, 0.0f), new Point3f(0.5f, 0.5f, 0.0f) };
    int[] indices = { 0, 1, 12, 11, 1, 2, 4, 3, 5, 6, 8, 7, 9, 10, 13, 12 };
    int[] colorIndices = { 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0};

    IndexedQuadArray array = new IndexedQuadArray(14, QuadArray.COLOR_3 |
            QuadArray.COORDINATES, 16);
    Color3f[] color = {fun.Cyan};
    array.setColors(0, color);
    array.setColorIndices(0, colorIndices);
    array.setCoordinates(0, pts);
    array.setCoordinateIndices(0, indices);

    Appearance app = new Appearance();
    PolygonAttributes plyAtt = new PolygonAttributes();
                // POLYGON_FILL POLYGON_LINE POLYGON_POINT
    plyAtt.setPolygonMode(PolygonAttributes.POLYGON_FILL);
                        // CULL_NONE CULL_FRONT CULL_BACK
    plyAtt.setCullFace(PolygonAttributes.CULL_NONE);
    app.setPolygonAttributes(plyAtt);
                        // an "E" letter with appearance
    return new Shape3D(array, app);
}
```

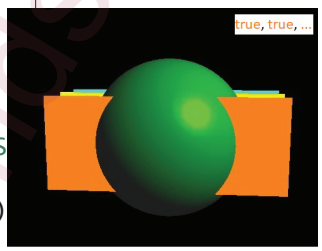


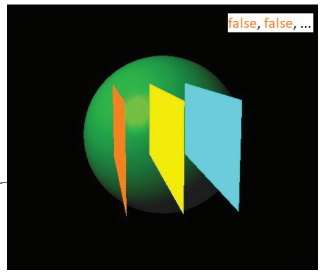
- The setting of coordinate colors overrides coloring attributes or a material's diffuse color
  - Color can be set individually for coordinates
  - Colors can be set for indexed geometry with color indices as in the previous slide
- Coordinate colors are interpolated along lines or across polygons





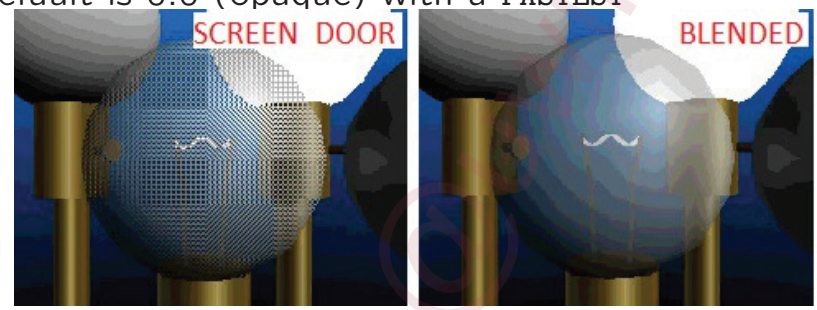
```
private Shape3D threePlanes() {
    Shape3D shape3D = new Shape3D();

    shape3D.addGeometry(quadGeo(0.2f, 1.0f, fun.Orange));
    shape3D.setGeometry(quadGeo(0.0f, 0.8f, fun.Yellow));
    shape3D.addGeometry(quadGeo(-0.2f, 0.3f, fun.Lime));

    Appearance look = new Appearance();
    PolygonAttributes pa = new PolygonAttributes();
    pa.setCullFace(PolygonAttributes.CULL_NONE);

    RenderingAttributes ra = new RenderingAttributes(
            // false, false, 0.0f, RenderingAttributes.ALWAYS
            true, true, 0.0f, RenderingAttributes.ALWAYS,
            true, false, false, RenderingAttributes.ROP_COPY);
    look.setPolygonAttributes(pa);
    look.setRenderingAttributes(ra);

    shape3D.setAppearance(look);

    return shape3D;
}
```



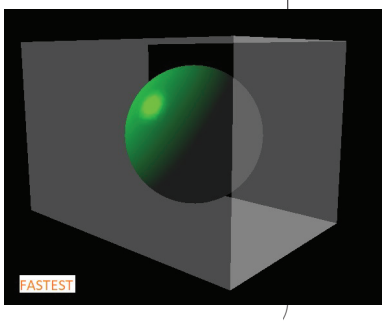



```
private Geometry quadGeo(float z, float alpha, Color3f clr) {
    Point3f[] verts = {new Point3f(-0.5f, 0.2f, z), new Point3f(-0.5f, -0.2f, z),
            new Point3f(0.5f, -0.2f, z), new Point3f(0.5f, 0.2f, z) };
    Color3f[] colors = { clr, clr, clr, clr };

    QuadArray rect = new QuadArray(4, QuadArray.COORDINATES | QuadArray.COLOR_3);
    rect.setCoordinates(0, verts);
    rect.setColors(0, colors);

    return rect;
}

private void createContent(TransformGroup scene_TG) {
    AppearanceExtra app_Extra = new AppearanceExtra();
    scene_TG.addChild(app_Extra.materializeSphere(0.35f));// add a shining ball
    app_Extra.addLights(scene_TG);                        // add lights

    // When 'false'-'false' in threePlanes(), the later child blocks previous
    scene_TG.addChild(threePlanes());                     // add three planes
}
```

- Color and transparency control:
  - `ColoringAttributes` controls intrinsic color
    - Use coloring attributes when a shape is not shaded
      - bright color for emissive points, lines, and polygons
      - helps to avoid expensive shading calculations
  - Transparency range is 0.0 (opaque) to 1.0 (invisible)
    - Mode: `SCREEN_DOOR`, `BLENDED`, `NONE`, `FASTEST`, and `NICEST`
    - Default is 0.0 (opaque) with a `FASTEST`

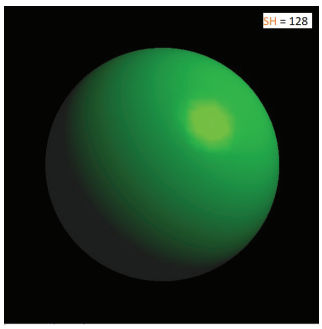


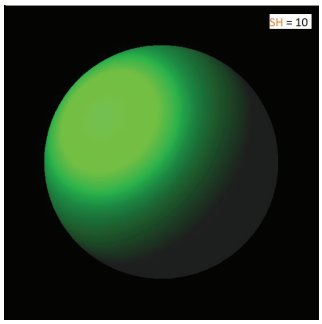

```
private Shape3D TransparentBox() {
    float c = 0.7f, d = -0.5f, e = 0.4f;
    Point3f[] q_verts = { new Point3f(-e, e, c), new Point3f(-e, -e, c),
            new Point3f(e, -e, c), new Point3f(e, e, c), new Point3f(-e, e, d),
            new Point3f(-e, -e, d), new Point3f(e, -e, d), new Point3f(e, e, d)};
    int[] q_indices = {0, 1, 2, 3, 4, 5, 6, 7, 5, 6, 2, 1, 4, 0, 3, 7};

    IndexedQuadArray qa = new IndexedQuadArray(8, QuadArray.COORDINATES, 16);

    Appearance app = new Appearance();
    ColoringAttributes ca = new ColoringAttributes(fun.White,
            ColoringAttributes.FASTEST);
    app.setColoringAttributes(ca);
    TransparencyAttributes ta = new TransparencyAttributes(
            TransparencyAttributes.FASTEST, 0.7f);
            // FASTEST NICEST SCREEN_DOOR BLENDED NONE
    app.setTransparencyAttributes(ta);
    PolygonAttributes pa = new PolygonAttributes();
    pa.setCullFace(PolygonAttributes.CULL_NONE);
    app.setPolygonAttributes(pa);

    qa.setCoordinates(0, q_verts);
    qa.setCoordinateIndices(0, q_indices);
    return new Shape3D(qa, app);
}
```



- Use materials when a shape is shaded
  - `Material` controls ambient, emissive, diffuse, specular color, and shininess factor
  - It overrides `ColoringAttributes` intrinsic color (when lighting is enabled)
  - An example of material colors


```
public class AppearanceExtra {
    /* a function to add a point and an ambient light to scene_TG */
    public void addLights(TransformGroup scene_TG) {
    /* a function to create and return material definition */
    public Material setMaterial() {
        int SH = 128;               // 10
        Material ma = new Material();
        ma.setAmbientColor(new Color3f(0.6f, 0.6f, 0.6f));
        ma.setEmissiveColor(new Color3f(0.0f, 0.0f, 0.0f));
        ma.setDiffuseColor(new Color3f(0.6f, 0.6f, 0.6f));
        ma.setSpecularColor(new Color3f(1.0f, 1.0f, 1.0f));
        ma.setShininess(SH);
        ma.setLightingEnable(true);
        return ma;
    }
    /* a function to create and return a sphere in 'radius' */
    public Sphere materializeSphere(float radius) {
        Appearance app = new Appearance();

        Color3f black = new Color3f(0f, 0f, 0f);
        ColoringAttributes ca = new ColoringAttributes(black,
                ColoringAttributes.NICEST);
        app.setColoringAttributes(ca);
        PolygonAttributes pa = new PolygonAttributes();
        app.setPolygonAttributes(pa);
        app.setMaterial(setMaterial());
        return new Sphere(radius, Sphere.GENERATE_NORMALS, 80, app);
}   }
```





PART II. MAPPING TEXTURE

1. The appearance of textures
   - Texture image colors can replace, modulate, or blend with shape color
     - Different *texture modes* are useful for different effects
   - Different texture images can be used at different distances between the shape and the user
     - Use lower resolution images for distant shapes
2. Combining texture and shape colors
   - A texture image may contain:
     - A red-green-blue color at each pixel
     - A transparency, or *alpha* value at each pixel

   - Alpha blending is a linear blending from one value to another as alpha goes from 0.0 to 1.0:

$$Value = (1.0 - alpha) * Value0 + alpha * Value1$$

3. *Texture mode* controls how texture affect shape color
   - Different texture modes in `TextureAttributes`
     - `REPLACE`: at each pixel, texture color completely replaces the shape's material color
     - `DECAL`: at each pixel, texture color is blended as a decal on top of the shape's material color
     - `MODULATE`: at each pixel, texture color modulates (filters) the shape's material color
     - `BLEND`: at each pixel, texture color blends the shape's material color with an arbitrary *blend color*

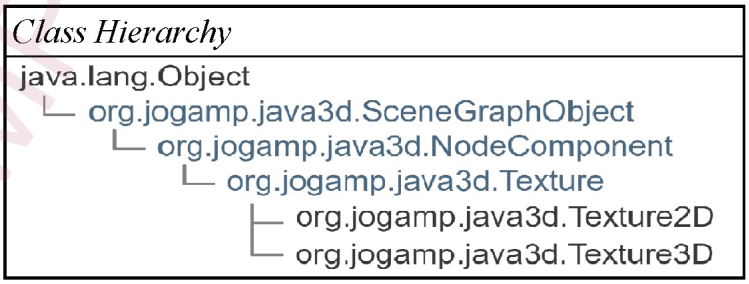
- Resulting appearance

Mode	Result color	Result transparency
`REPLACE`	T_{rgb}	T_a
`DECAL`	$S_{rgb} * (1 - T_a) + T_{rgb} * T_a$	S_a
`MODULATE`	$S_{rgb} * T_{rgb}$	$S_a * T_a$
`BLEND`	$S_{rgb} * (1 - T_{rgb}) + B_{rgb} * T_{rgb}$	$S_a * T_a$

  - T_{rgb} and T_a are texture's pixel color and alpha
  - S_{rgb} and S_a are the shape's color and alpha
  - B_{rgb} and B_a are the shape blend color and alpha
- Typical use:
  - Use `REPLACE` for emissive, glowing "neon" textures
    - Textures where lighting is painted in

  - Use `MODULATE` on a white shape for shaded textures
    - Most textured shaded surfaces
  - Use `BLEND` on a colored shape for colorized textures
    - Colorizing a grayscale woodgrain, marble, etc.

4. Texture control attributes include a few node components
   - `Texture` controls mapping attributes, and is the base class of two `NodeComponents` for image selection



5. Simple texture recipe

(1) Prepare texture images

- Use picture formats such as gif, jpg, and PNG.
- Check dimensions to avoid runtime exception
  - For rendering efficiency, the sides of the image must be integer powers of two, such as 128×256.

(2) Load the texture

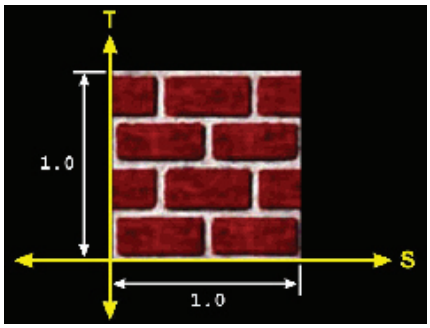
- Use `TextureLoader` to load file
- Use `ImageComponent` to hold the loaded image
  - The extended `ImageComponent2D` holds a 2D image.

(3) Set the texture in `Appearance` bundle

(4) Specify `TexturCoordinates` of Geometry


```
public static Texture texturedApp(String name) {
    String filename = "images/" + name + ".jpg";
    TextureLoader loader = new TextureLoader(filename, null);
    ImageComponent2D image = loader.getImage();
    if (image == null)
        System.out.println("Cannot load file: " + filename);

    Texture2D texture = new Texture2D(Texture.BASE_LEVEL,
            Texture.RGBA, image.getWidth(), image.getHeight());
    texture.setImage(0, image);

    return texture;
}

public Sphere textureSphere(float rad, int div) {
    Appearance app = new Appearance();
    app.setTexture(texturedApp("earth"));
    Sphere textured_sphere = new Sphere(rad,
            Primitive.GENERATE_TEXTURE_COORDS, div, app);

    return textured_sphere;
}
```

6. *Texture coordinates* describe a 2D shape that maps from parts of a texture to parts of a shape.
   - Texture images have a *true size* and a *logical size*
     - True size is the image's width and height in pixels
     - Logical size is a generic treatment of image dimensions, which has always a width of 1.0 and a height of 1.0



7. Mapping with texture coordinates and `TextureAttributes`

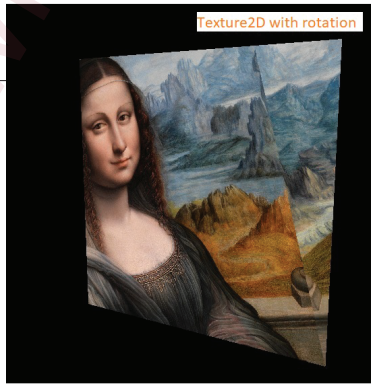
   (1) Create lists of 3D coordinates, (lighting normals,) and texture coordinates for the vertices

   (2) Create a `QuadArray` and set the vertex coordinates, (lighting normals,) and texture coordinates

   (3) Specify `TextureAttributes` in the `Appearance` bundle
   - Create `TextureAttributes`
   - Perform transformation(s) with `Transform3D`
   - Set the transformation with `setTextureTransform`

   (4) Follow the normal procedure to assemble the shape with its geometry and appearance as in Slide 17.

```
private Shape3D texturePlane(float w, float h, float s) {
    Point3f[] verts = { new Point3f(-w * s, -h * s, 0f),
            new Point3f(w * s, -h * s, 0f), new Point3f(w * s, h * s, 0f),
            new Point3f(w * -s, h * s, 0f) };

    QuadArray qa = new QuadArray(4, GeometryArray.COORDINATES |
            GeometryArray.COLOR_3 | GeometryArray.TEXTURE_COORDINATE_2);

    qa.setCoordinates(0, verts);
    float uv0[] = { 0f, 0f };
    float uv1[] = { 1f, 0f };
    float uv2[] = { 1f, 1f };
    float uv3[] = { 0f, 1f };
    qa.setTextureCoordinate(0, 0, uv0);
    qa.setTextureCoordinate(0, 1, uv1);
    qa.setTextureCoordinate(0, 2, uv2);
    qa.setTextureCoordinate(0, 3, uv3);

    Shape3D shape3D = new Shape3D(qa, texturedApp());
    return shape3D;
}
```




```
private Texture setTexture() {
    String filename = "images/mona_big.jpg";
    TextureLoader loader = new TextureLoader(filename, null);
    ImageComponent2D image = loader.getImage();
    if (image == null)
        System.out.println("load failed for texture: " + filename);

    Texture2D texture = new Texture2D(Texture.BASE_LEVEL,
            Texture.RGBA, image.getWidth(), image.getHeight());
    texture.setImage(0, image);

    return texture;
}
```

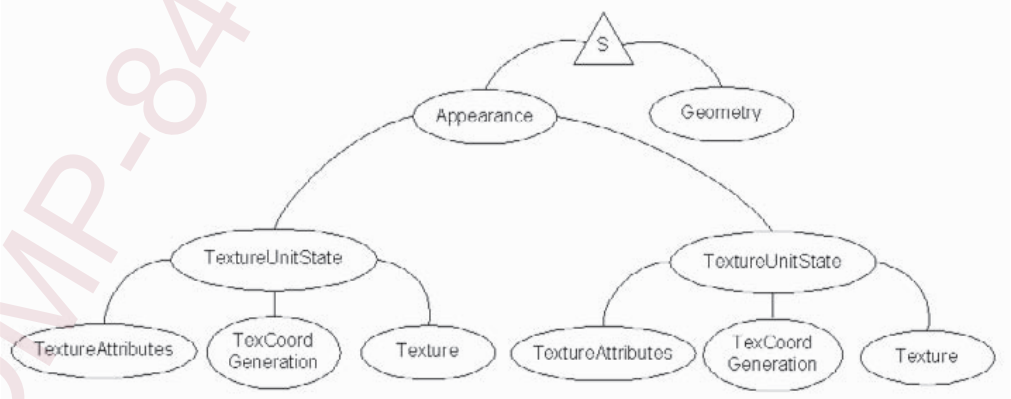
```
private Appearance texturedApp() {
    Appearance appear = new Appearance();

    appear.setTexture(setTexture());

    PolygonAttributes polyAttrib = new PolygonAttributes();
    polyAttrib.setCullFace(PolygonAttributes.CULL_NONE);
    appear.setPolygonAttributes(polyAttrib);

    TextureAttributes ta= new TextureAttributes();
    ta.setTextureMode(TextureAttributes.REPLACE);
    appear.setTextureAttributes(ta);

    int angle = 0; // 15
    Vector3d scale = new Vector3d(1f, 1f, 1f);  // 0.75
    Transform3D transMap = new Transform3D();
    transMap.rotZ((angle / 90.0f) * Math.PI/2);
    transMap.setScale(scale);
    ta.setTextureTransform(transMap);

    return appear;
}
```

- More on `TextureAttributes` and `TexCoordGeneration`

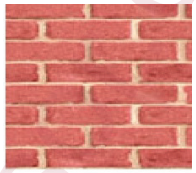
```
private Sphere textureSphere(AppearanceExtra app_Extra, float rad, int div) {
    Appearance app = new Appearance();
                                        // BLEND MODULATE REPLACE DECAL
    TextureAttributes ta = new TextureAttributes();
    ta.setTextureMode(TextureAttributes.REPLACE);
    app.setTextureAttributes(ta);

                // OBJECT_LINEAR EYE_LINEAR SPHERE_MAP NORMAL_MAP REFLECTION_MAP
    TexCoordGeneration tcg = new TexCoordGeneration(TexCoordGeneration.SPHERE_MAP,
            TexCoordGeneration.TEXTURE_COORDINATE_3);
    app.setTexCoordGeneration(tcg);

    app.setTexture(setTexture());
    app.setMaterial(app_Extra.setMaterial());

    Sphere textured_sphere = new Sphere(rad,
            Primitive.GENERATE_TEXTURE_COORDS |
            Primitive.GENERATE_NORMALS, div, app);

    return textured_sphere;
}
```

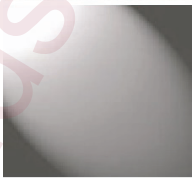
8. Multitexture

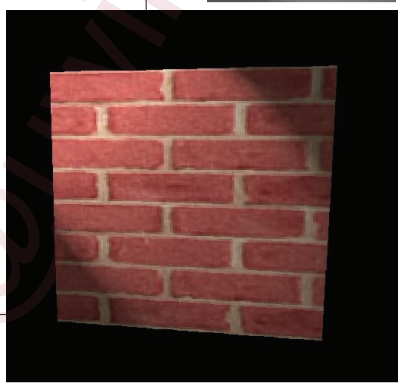
- Multitexturing applies different textures to one geometrical object in a selective or combinational manner to create interesting visual effects.



```
private Appearance createAppearance() {
    Appearance Appear = new Appearance();

    TextureUnitState[] array = new TextureUnitState[2];
    TexCoordGeneration tcg = new TexCoordGeneration();
    tcg.setEnable(false);     // use the texture coordinates

    TextureAttributes ta = new TextureAttributes();
    ta.setTextureMode(TextureAttributes.MODULATE);
                          // BLEND MODULATE REPLACE DECAL
    array[0] = texState("light.jpg", ta, tcg);
    array[1] = texState("bricks.jpg", ta, tcg);

    PolygonAttributes polyAttrib = new PolygonAttributes();
    polyAttrib.setCullFace(PolygonAttributes.CULL_NONE);
    Appear.setPolygonAttributes(polyAttrib);

    Appear.setTextureUnitState(array);
    return Appear;
}

private Shape3D DualTexturedWall() {
    Shape3D dualWall = new Shape3D();
    dualWall.setGeometry(createGeometry());
    dualWall.setAppearance(createAppearance());
    return dualWall;
}
```








```
private Geometry createGeometry() {
    int[] setmap = { 0, 0 };        // 2 textures, both using texture coordinate set 0
    QuadArray plane = new QuadArray(4, GeometryArray.COORDINATES |
            GeometryArray.TEXTURE_COORDINATE_2, 1, setmap);
    Point3f[] verts = {new Point3f(-1.0f, 1.0f, -1.0f), new Point3f(-1.0f, -1.0f, -1.0f),
            new Point3f(1.0f, -1.0f, -1.0f), new Point3f(1.0f, 1.0f, -1.0f)};
    plane.setCoordinates(0, verts);
    TexCoord2f q = new TexCoord2f(0.0f, 1.0f);
    plane.setTextureCoordinate(0, 0, q);
    q.set(0.0f, 0.0f); plane.setTextureCoordinate(0, 1, q);
    q.set(1.0f, 0.0f); plane.setTextureCoordinate(0, 2, q);
    q.set(1.0f, 1.0f); plane.setTextureCoordinate(0, 3, q);
    return plane;
}

private TextureUnitState texState(String fn, TextureAttributes ta, TexCoordGeneration tcg) {
    String filename = "images/" + fn;
    TextureLoader loader = new TextureLoader(filename, null);
    ImageComponent2D image = loader.getImage();
    if (image == null)
        System.out.println("load failed for texture: " + filename);
    Texture2D texture = new Texture2D(Texture.BASE_LEVEL,
            Texture.RGBA, image.getWidth(), image.getHeight());
    texture.setImage(0, image);

    TextureUnitState tt_State = new TextureUnitState(texture, ta, tcg);
    tt_State.setCapability(TextureUnitState.ALLOW_STATE_WRITE);
    return tt_State;
}
```

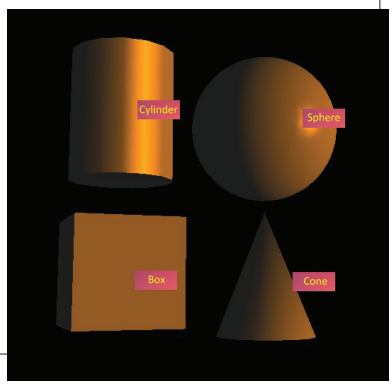
9. Textured (external) object

```
private Appearance setApp(Color3f clr) {
    Appearance app = new Appearance();
    app.setMaterial(Commons.setMaterial(clr));
    ColoringAttributes colorAtt = new ColoringAttributes();
    colorAtt.setColor(clr);
    app.setColoringAttributes(colorAtt);
    app.setTexture(AppearanceTexture2D.texturedApp("penguin"));
    return app;
}

private TransformGroup loadShape(Vector3d scale) {
    Scene s = ShapeObjLoad.loadShape("penguin");        // load th

    BranchGroup objBG = s.getSceneGroup();              // retriev
    Shape3D pngnShape = (Shape3D) objBG.getChild(0);
    pngnShape.setAppearance(setApp(Commons.Orange));

    TransformGroup comp_TG = scaleTG(scale);
    comp_TG.addChild(objBG);                            // attach
    return comp_TG;
}
```

10. Raster geometry

- A `raster` positions a 2D image in the 3D scene
  - It anchors the 2D image to a 3D point
  - Image's size is independent to the viewer's distance
  - Useful for annotation text, sprites, etc.

```
private static void labelledShape(TransformGroup shapeTG, Primitive prm, String fn, float d) {

    String filename = "images/lable_" + fn + ".jpg";
    TextureLoader myLoader = new TextureLoader( filename, null );
    ImageComponent2D myLabel = myLoader.getImage( );

    Raster myRaster = new Raster( );
    myRaster.setPosition( new Point3f( 0.0f, 0.0f, d));
    myRaster.setImage( myLabel );
    myRaster.setSize( 60, 27 );
    Shape3D myShape = new Shape3D( myRaster);
    shapeTG.addChild(myShape);
    shapeTG.addChild(prm);
}
```



```
private static BranchGroup labelledPrimitives() {
    Vector3f[] position = {new Vector3f(0.3f, 0.3f, -0.3f), new Vector3f(-0.3f, 0.3f, -0.3f),
            new Vector3f(-0.3f, -0.3f, -0.3f), new Vector3f(0.3f, -0.3f, -0.3f)};
    String[] lable = {"sphere", "cylinder", "box", "cone"};
    float[] dist = {0.35f, 0.25f, 0.25f, 0.25f};

    Appearance app = new Appearance();
    app.setMaterial(AppearanceExtra.setMaterial(CommonsXY.Orange));

    Primitive[] priShape = new Primitive[4];
    priShape[0] = new Sphere(0.3f, Sphere.GENERATE_NORMALS, 60, app);
    priShape[1] = new Cylinder(0.2f, 0.5f, app);
    priShape[2] = new Box(0.2f, 0.2f, 0.2f, app);
    priShape[3] = new Cone(0.2f, 0.5f, app);

    BranchGroup objBG = new BranchGroup();
    Transform3D trans3d = new Transform3D();
    TransformGroup trans = null;
    for (int i = 0; i < 4; i++) {
        trans3d.setTranslation(position[i]);
        trans = new TransformGroup(trans3d);
        labelledShape(trans, priShape[i], lable[i], dist[i]);
        objBG.addChild(trans);
    }
    return objBG;
}
```